

Anonymous Router Network

Honors Oral Defense

Committee: Professor Ke Wu
Professor Paul Grubbs

by
Anthony Flath
April 3rd, 2026

Roadmap

- Preliminaries and Overview
- Security Definitions
- Relevant Schemes/Algorithms
- Our Construction
- Results and Benchmarks
- Conclusion

Roadmap

- **Preliminaries and Overview**
- Security Definitions
- Relevant Schemes/Algorithms
- Our Construction
- Results and Benchmarks
- Conclusion

Anonymous Communication

- Popular forms include threshold security, namely Tor
- Not necessarily bad
- Drawbacks
 - Actors with resources can gain significant influence
 - Can often be slow

Non-Interactive Anonymous Router

- “NIAR” by Shi and Wu
- A single untrusted router with similar results
- Key takeaway: oblivious shuffling
- Theoretical, not implemented

(somewhat) Practical Anonymous Router

- “sPAR” by Das and Park
- Similar idea to NIAR, and showed implementation
- However, relatively slow
 - Needs at least 4 seconds for 32 messages transmitted
 - Speed scales with respect to the amount of messages

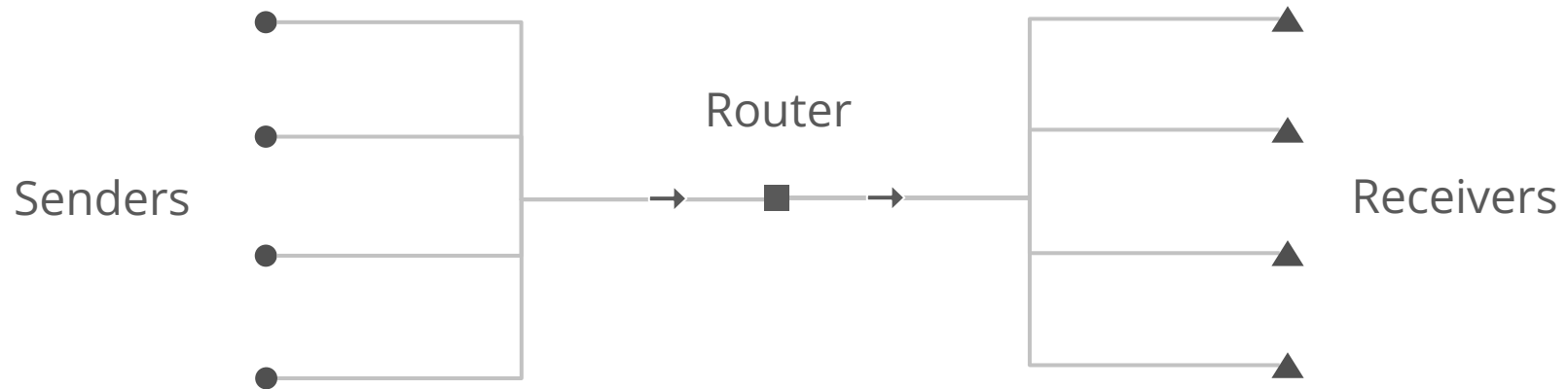
Motivation for our work

- Is it possible to achieve a fast implementation?
- Can we limit scaling performance with respect to number of messages?
- Is it possible to eliminate reliance on a single machine?

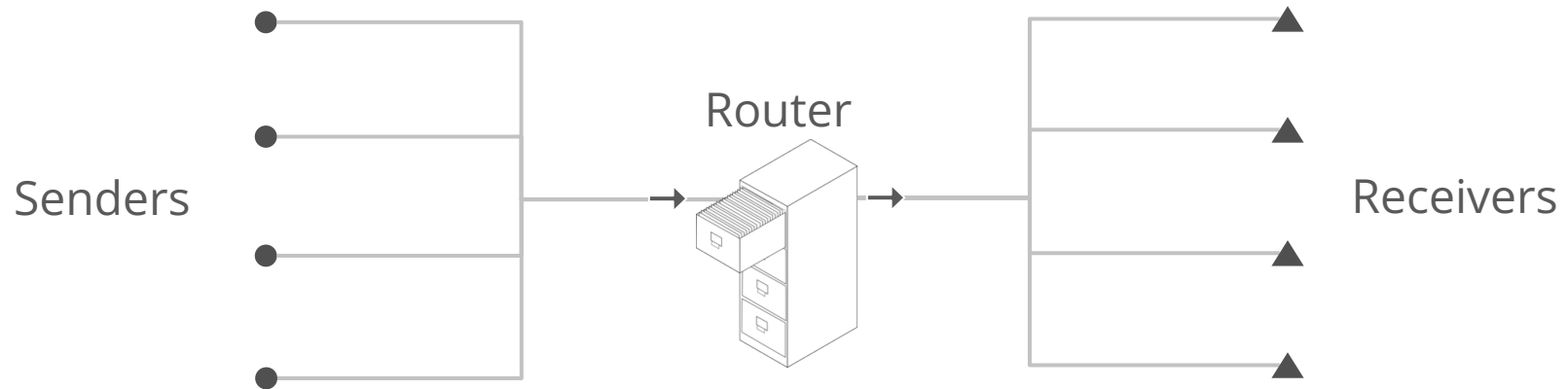
Our results

- Adding more routers to the system improves performance.
- A subset of messages still get sent when some routers fail.
- Opens the door for future performance improvements.

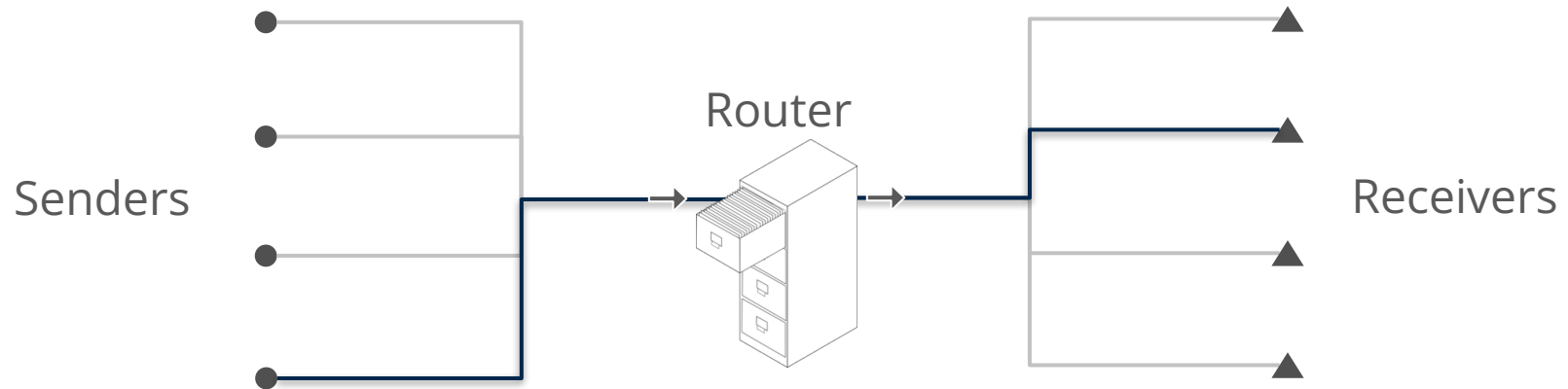
Informal overview of the system



Informal overview of the system

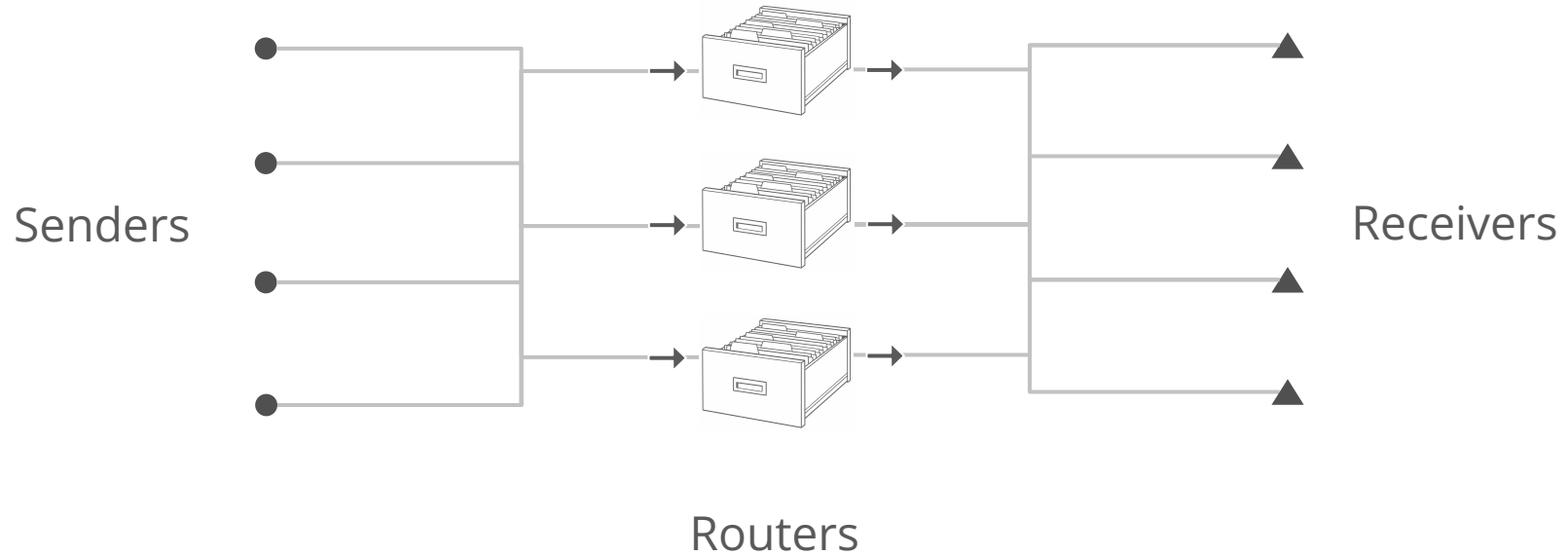


Informal overview of the system



Big filing cabinets take a while to organize!

Informal overview of the system



Less informal overview

n	number of senders/receivers
k	number of routers
p	bin multiplier
π	message destination
C	ciphertext array
$\mathcal{M}$	message multiset

- $(\widehat{\mathbf{pk}}, \text{user keys}) \leftarrow \mathbf{Setup}(1^\lambda, n, k, p)$
- $(\{c_j\}_{j \in [k]}) \leftarrow \mathbf{Enc}(\widehat{\mathbf{pk}}, m, \pi)$
- $(C) \leftarrow \mathbf{Rte}(\{c_i\}_{i \in [n]})$
- $(\mathcal{M}) \leftarrow \mathbf{Dec}(\text{user keys}, C)$

Roadmap

- Preliminaries and Overview
- **Security Definitions**
- Relevant Schemes/Algorithms
- Our Construction
- Results and Benchmarks
- Conclusion

The two notions of security

- Sender Anonymity
 - a. The sender of a message are not known
 - b. *All messages are visible!*
- Receiver Anonymity
 - a. The sender **and** receiver of a message are not known
 - b. Only the receiver and the sender can see a message

The ARN experiment

- $n, k, p, \mathcal{K} \leftarrow \mathcal{A}(1^\lambda)$
- $\widehat{\mathbf{pk}}, \text{ user keys} \leftarrow \mathbf{Setup}(1^\lambda, n, k)$
- For polynomial number of rounds:

- $\{(\pi^b, m^b) \text{ per user}\}_{b \in \{0,1\}}, \mathcal{K}'\text{'s ciphertexts} \leftarrow \mathcal{A}(\widehat{\mathbf{pk}}, \text{ corrupt user keys})$
- For all honest senders i : $\{c_{i,j}\}_{j \in [k]} \leftarrow \mathbf{Enc}(\widehat{\mathbf{pk}}, m_i^b, \pi_i^b)$
- For all routers j : $C_j \leftarrow \mathbf{Rte}(\{c_{i,j}\}_{i \in [n]})$
- For all routers j : $\mathcal{M}_j \leftarrow \mathbf{Dec}(\text{user keys}, C_j)$

n	number of senders/receivers
k	number of routers
p	bin multiplier
π	message destination
C	ciphertext array
$\mathcal{M}$	message multiset

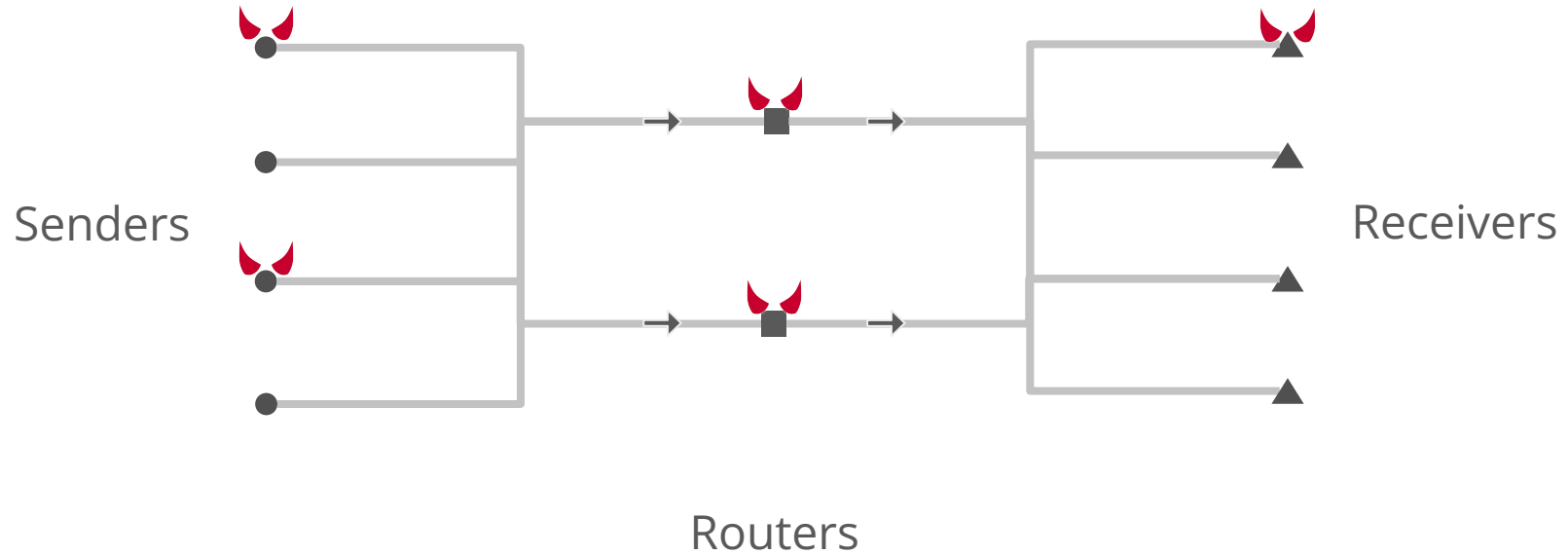
The ARN experiment correctness

- In both worlds:
 - The same amount of information is leaked
 - The content of all messages sent by honest senders is the same
 - Corrupt receivers send the same message

Sender Anonymity

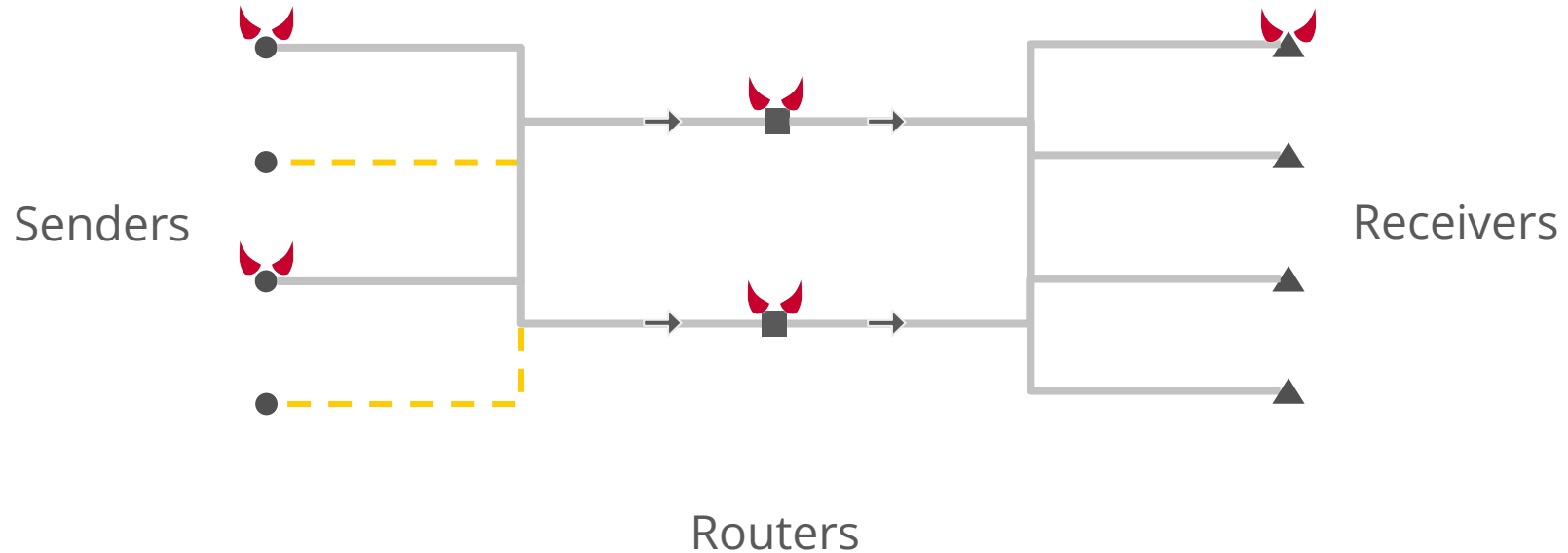
- An adversary **can** know:
 - The destination and message from every corrupt sender
 - The content of every message
- An adversary **cannot** know:
 - The sender of a specific message
 - The destination given by a sender

Sender Anonymity example



Sender Anonymity example

	Known by adversary
	Not known by adversary

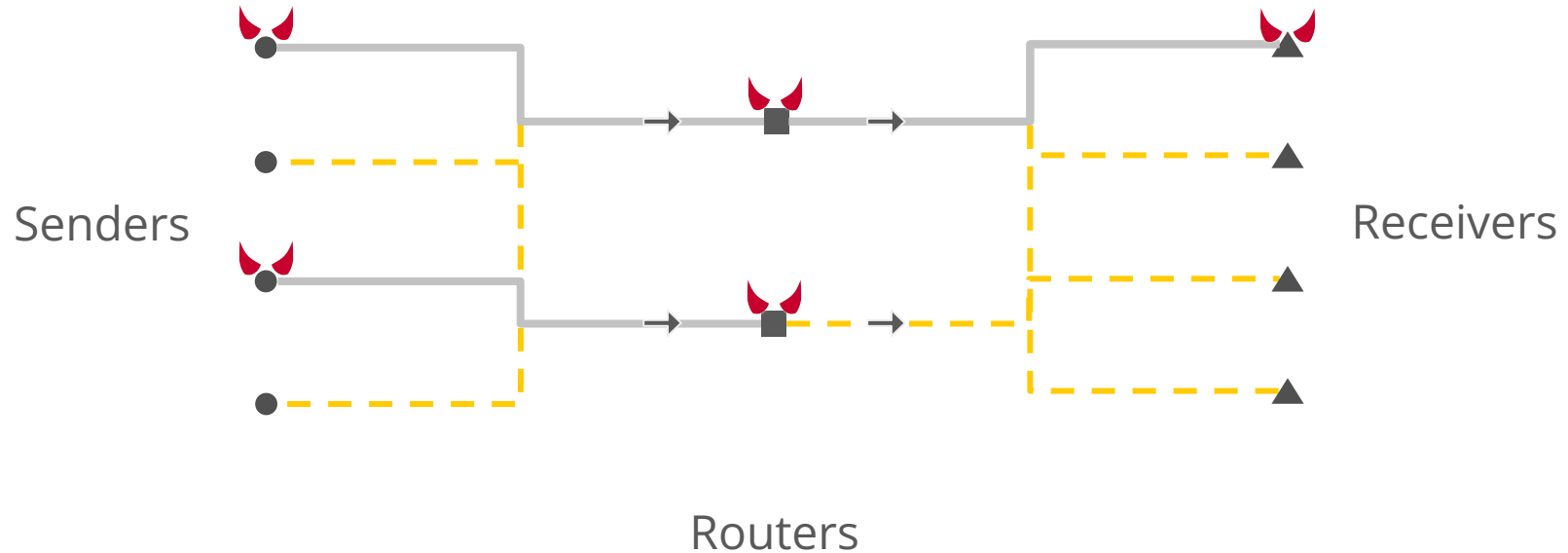


Receiver Anonymity

- An adversary can know:
 - All corrupt-to-corrupt communication
 - The content of every message sent to a corrupt receiver
- An adversary cannot know:
 - Everything else regarding the content and mapping of messages

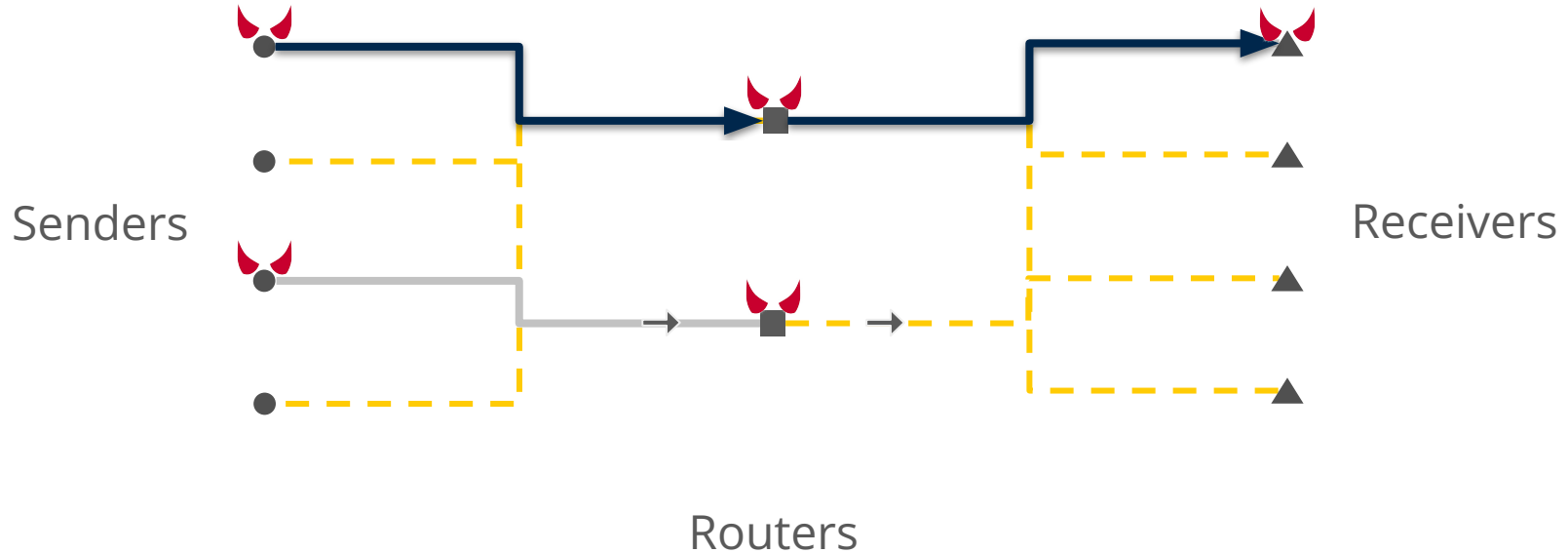
Receiver Anonymity example

	Known by adversary
	Not known by adversary



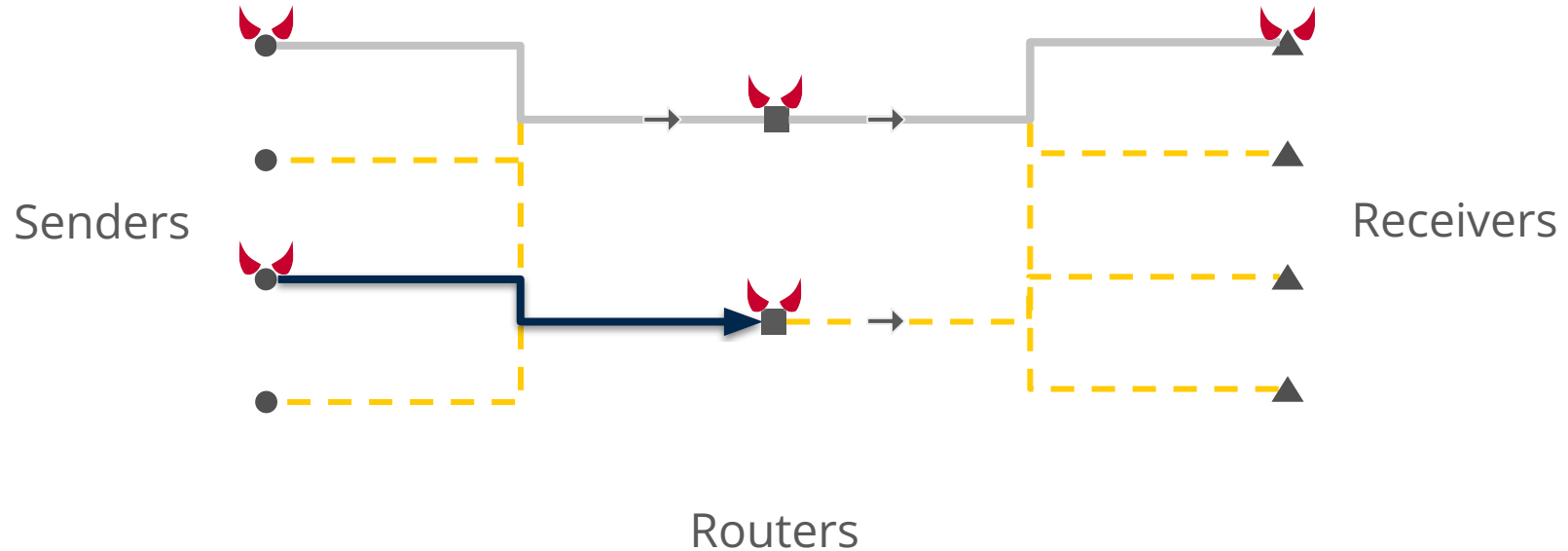
Receiver Anonymity example

	Known by adversary
	Not known by adversary



Receiver Anonymity example

	Known by adversary
	Not known by adversary



Roadmap

- Preliminaries and Overview
- Security Definitions
- **Relevant Schemes/Algorithms**
- Our Construction
- Results and Benchmarks
- Conclusion

Fully homomorphic encryption (FHE)

- Operate (addition and multiplication) on ciphertexts
- This is the basis of the oblivious placing mechanism

$$m_1 + m_2 = \text{Dec}(\text{Enc}(m_1) + \text{Enc}(m_2))$$

$$m_1 \cdot m_2 = \text{Dec}(\text{Enc}(m_1) \cdot \text{Enc}(m_2))$$

Why this is useful

- Suppose we wanted to place

$$m_1 \cdot 1 = \text{Dec}(\text{Enc}(m_1) \cdot \text{Enc}(1))$$

- Suppose we did not want to place

$$m_1 \cdot 0 = \text{Dec}(\text{Enc}(m_1) \cdot \text{Enc}(0))$$

FHE with Ring Learning With Errors (RLWE)

- A known polynomial multiplied by a secret polynomial added with some small error is indistinguishable from a randomly selected polynomial.

$$\text{polynomial} \times \text{secret} + \text{error} \equiv \text{random polynomial}$$

q	ciphertext modulus
$\mathcal{R}_q$	$\mathbb{Z}[x]/(x^N + 1) \bmod q, q, N \in \mathbb{N}$

For a secret $s \in \mathcal{R}_q$ with polynomials $a, u \in \mathcal{R}_q$ and small error e , $(a, a \cdot s + e)$ and (a, u) are computationally indistinguishable.

RLWE Ciphertext

q	ciphertext modulus
t	plaintext modulus $t \ll q$
$\mathcal{R}_q$	$\mathbb{Z}[x]/(x^N + 1) \bmod q, q, N \in \mathbb{N}$
χ	some small error distribution

$$\begin{aligned}\Delta &= q/t \\ m &\in \mathcal{R}_t \\ a, s &\in \mathcal{R}_q \\ e &\leftarrow \chi\end{aligned}$$

$\text{RLWE}_s(m) = (a, b) \in \mathcal{R}_q^2$ such that $b = a \cdot s + \Delta \cdot m + e$

$$\Delta m \approx b - a \cdot s$$

RLWE Addition

- RLWE + RLWE makes sense:

$$\text{RLWE}_s(m_1) + \text{RLWE}_s(m_2) = (a = a_1 + a_2, b = b_2 + b_2)$$

$$\begin{aligned}\Delta \cdot (m_1 + m_2) &\approx b - s \cdot a \\ &= b_1 + b_2 - s(a_1 + a_2) \\ &= (a_1 \cdot s + \Delta \cdot m_1 + e_1) + (a_2 \cdot s + \Delta \cdot m_2 + e_2) - s(a_1 + a_2) \\ &= \Delta \cdot m_1 + e_1 + \Delta \cdot m_2 + e_2\end{aligned}$$

- Multiplication is not so simple.

Ring Gentry-Sahai-Waters (RGSW)

- Vector of decomposed RLWE ciphertexts

$$\text{RGSW}_s(m) = (\text{Decompose}(\text{RLWE}_s(-s \cdot m)), \text{Decompose}(\text{RLWE}_s(m)))$$

$$\gamma = \left[\begin{array}{ccccccc} \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ q & \frac{q}{\beta} & \frac{q}{\beta^2} & \frac{q}{\beta^3} & \dots & \frac{q}{\beta^{\ell-1}} & 0 \end{array} \right]$$

Zama. Accessed 3-30-2026. <https://www.zama.org/post/tfhe-deep-dive-part-3>

- This is a nuanced topic that is largely outside of our scope.

Why RGSW is important

- RGSW allows for multiplication.

$$\begin{array}{l} m_1, m_2 \in \mathcal{R}_t \\ s \in \mathcal{R}_q \end{array}$$

$$\text{RLWE}_s(m_1) \cdot \text{RGSW}_s(m_2) = \text{RLWE}_s(m_1 \cdot m_2)$$

Multi-party FHE

Multi-party FHE

- We want a scheme in which a router cannot simply decrypt.
 - $(\widehat{\text{pk}}, \text{user secret keys}) \leftarrow \mathbf{Setup}(1^\lambda, n)$
 - $(c) \leftarrow \mathbf{Enc}(\widehat{\text{pk}}, m)$
 - $(m) \leftarrow \mathbf{Dec}(\text{user secret keys}, c)$

Multi-party FHE with RLWE - I

- Participants agree on some a .
- Each user i calculates $\text{pk} = a \cdot s_i + e$.
- $\widehat{\text{pk}} = \sum_{i=1}^n \text{pk}$.

Multi-party FHE with RLWE - II

- $(a', b') \leftarrow \mathbf{Enc}(\widehat{\mathbf{pk}}, m)$:
 - Sample u from $\mathcal{R}_q$
 - $a' = u \cdot a + e$ and $b' = u \cdot \widehat{\mathbf{pk}} + \Delta m + e$
- $(\Delta m) \leftarrow \mathbf{Dec}(\text{user secret keys}, (a', b'))$:
 - Each user i calculates $d_i = a' \cdot s_i + e_i$
 - $\Delta m = b' - \sum_{i=1}^n d_i$

Homomorphic Placing

Understanding the placing problem

- Reminder: we want the router to be able to obviously place ciphertexts.
- We could have RLWE encrypted messages, with RGSW encrypted bits.
- Have the user send the encrypted bits!

The placing mechanism

- SortingHat by Cong et al.
- Binary tree structure in which the the root is distributed to the leaves.
- Distribution is done according to the indices.

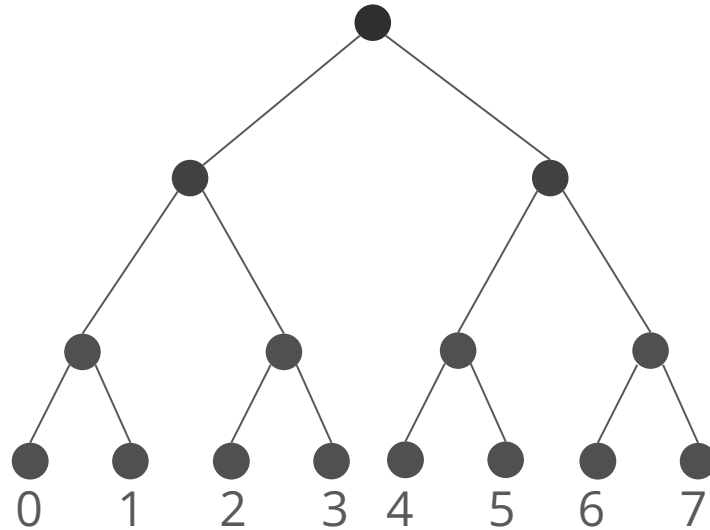
Quick detour: Plinko

- Plinko is from the game show “The Price is Right”



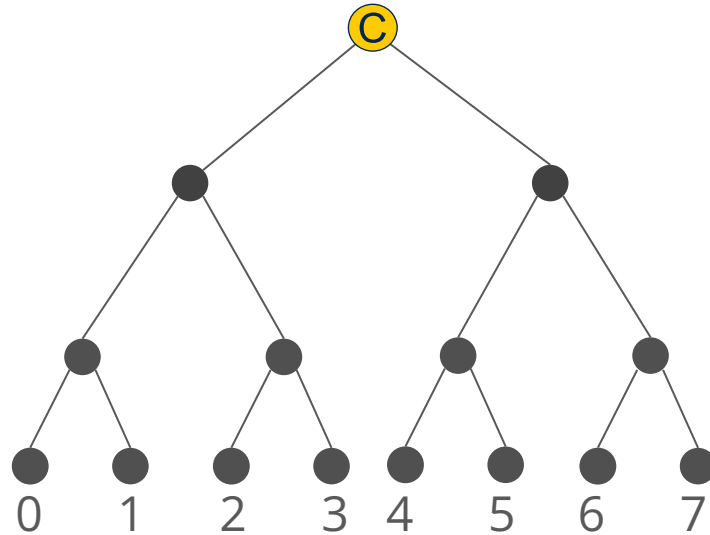
The placing mechanism through Plinko

Index: [1, 0, 1]



The placing mechanism through Plinko

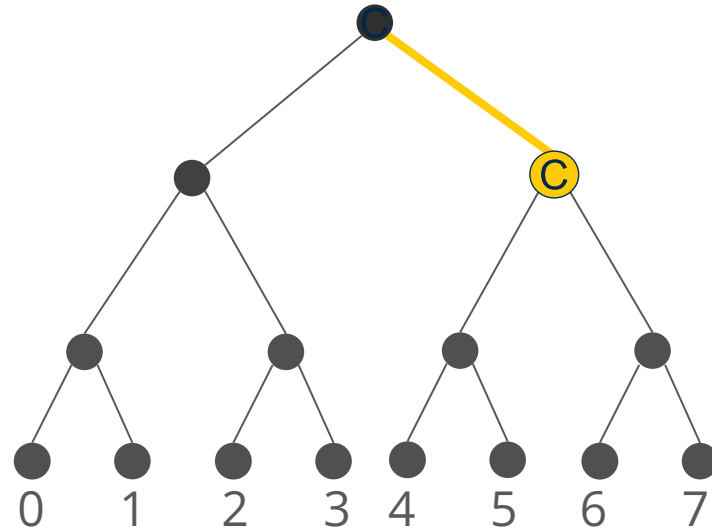
Index: [1, 0, 1]



1

The placing mechanism through Plinko

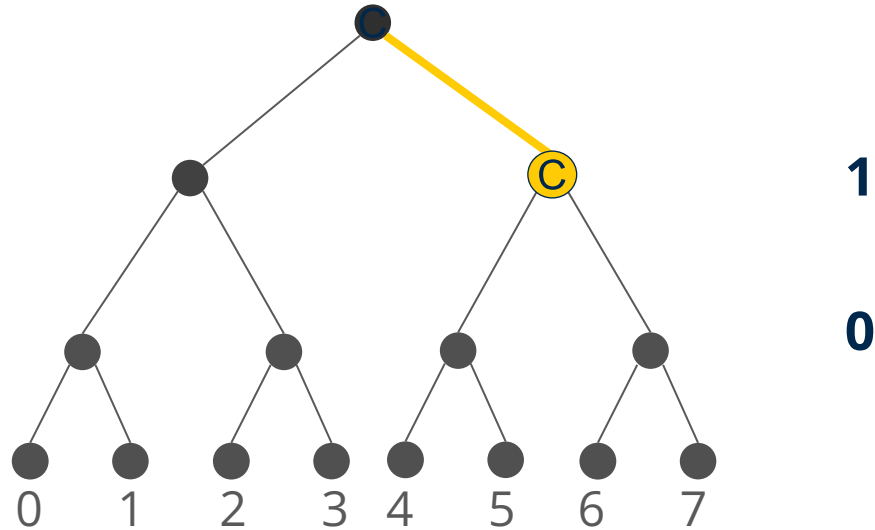
Index: [1, 0, 1]



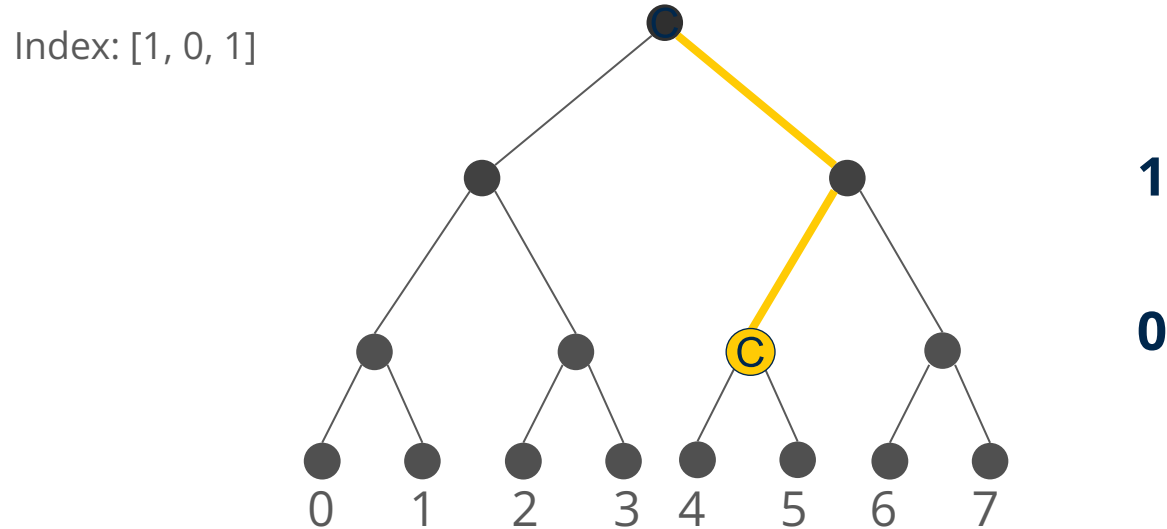
1

The placing mechanism through Plinko

Index: [1, 0, 1]

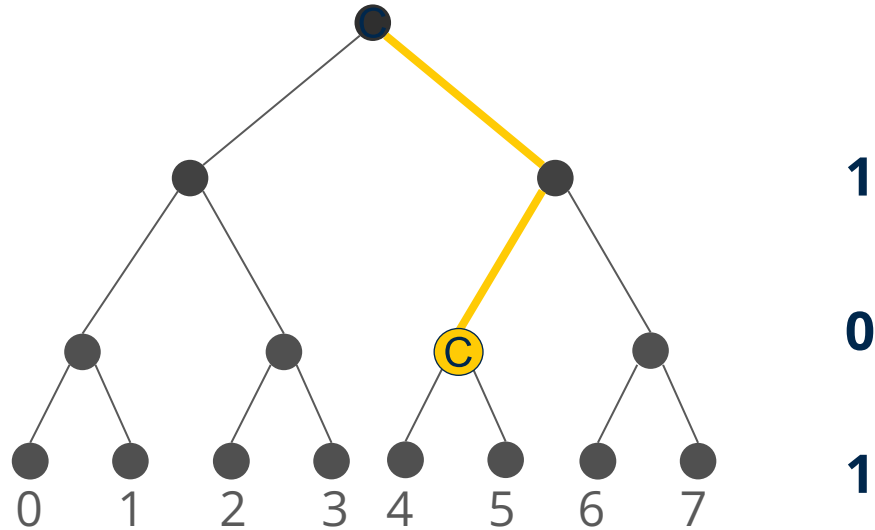


The placing mechanism through Plinko



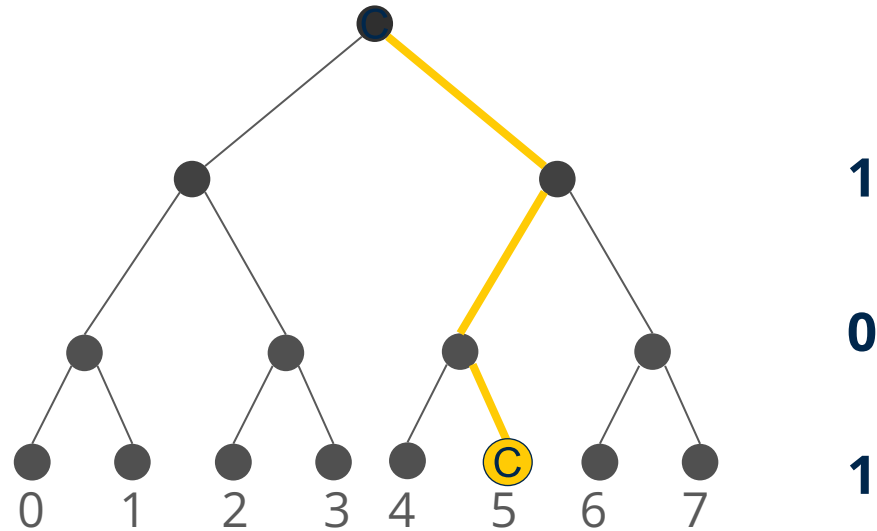
The placing mechanism through Plinko

Index: [1, 0, 1]



The placing mechanism through Plinko

Index: [1, 0, 1]



The actual placing algorithm

Algorithm 1 Homomorphic Placing

Input: $c := \text{RLWE}_{\widehat{\text{pk}}}(m)$, $A = \{\text{RGSW}_{\widehat{\text{pk}}}(m_w)\}_{w \in \{0, \dots, \log(p \cdot n) - 1\}}$

Output: RLWE ciphertext array with c placed according to A

```
1: function HOMPLACING( $c, A^*$ )
2:    $b_0 := c$ 
3:    $b_x := \text{RLWE}_{\widehat{\text{pk}}}(0)$  for  $x \in \{1, \dots, 2p \cdot n - 2\}$ 
4:   for  $w \leftarrow \{0, \dots, \log(p \cdot n) - 1\}$  do                                ▷ Iterate through each layer
5:     for  $z \leftarrow \{0, \dots, 2^w - 1\}$  do                                ▷ Iterate through each leaf
6:        $b_{2^{w+1}+2 \cdot z-1} := b_{2^w+z-1} - (b_{2^w+z} \cdot A_w^*)$ 
7:        $b_{2^{w+1}+2 \cdot z+1} := (b_{2^w+z-1} \cdot A_w^*)$ 
8:     end for
9:   end for
10:  return  $b_{p \cdot n}, \dots, b_{2p \cdot n-1}$ 
11: end function
```

What about the rest of the messages?

- We add the output of all placing algorithm executions.
- This results in a failure rate dependent on the value of the bin multiplier.
- We'll analyze this in the results section.

Roadmap

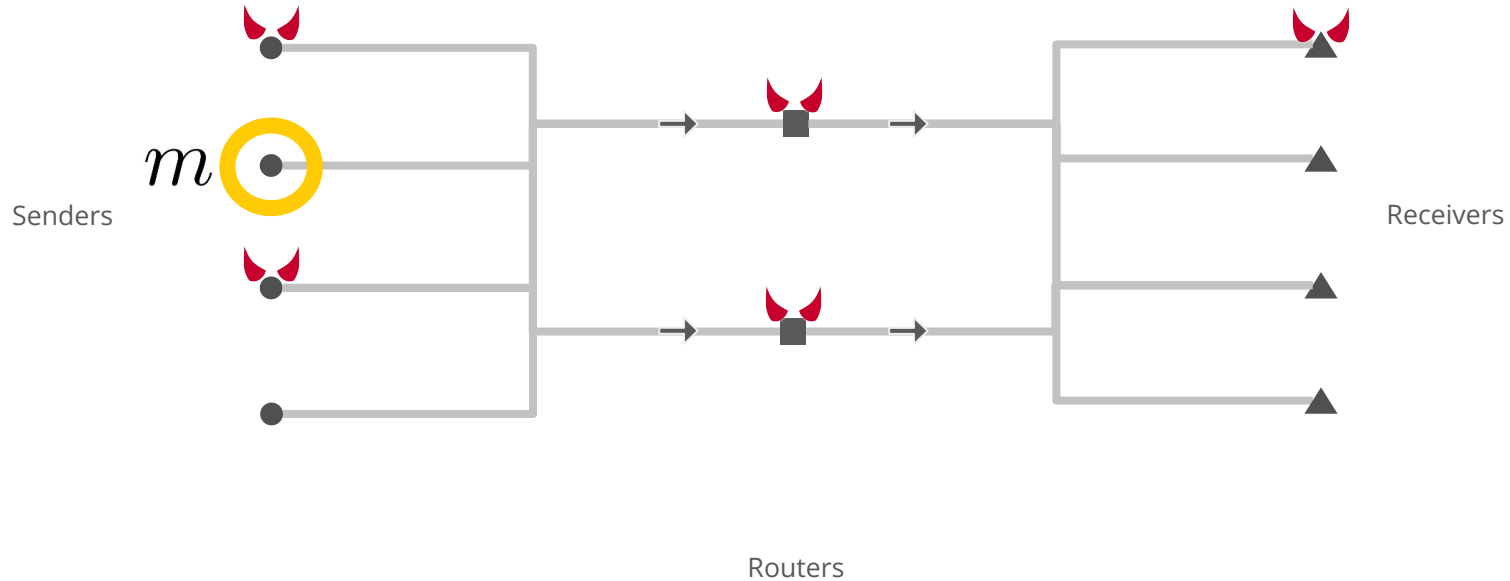
- Preliminaries and Overview
- Security Definitions
- Relevant Schemes/Algorithms
- **Our Construction**
- Results and Benchmarks
- Conclusion

ARN with Sender Anonymity

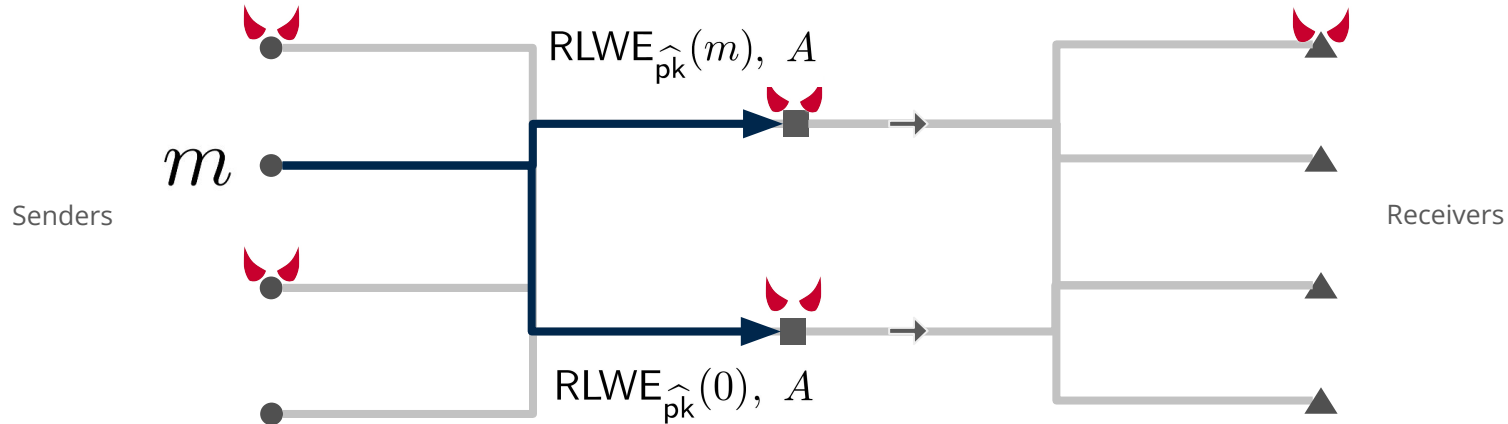
ARN with Sender Anonymity reminder

- We want messages to not be tied to its sender.
- We do not care if the message is seen by anyone.

ARN with Sender Anonymity example



ARN with Sender Anonymity example



$$\forall x \in [\log p \cdot n], A_x = \text{RGSW}_{\hat{pk}}(a_x) \text{ where } a \in \{0, 1\}$$

ARN with Sender Anonymity construction - I

- **Setup**($1^\lambda, n, k$): Output $\widehat{\text{pk}}$ and user secret keys from **MP.Setup**($1^\lambda, n, k$).
- **Enc**($\widehat{\text{pk}}, m, \pi$):
 - Randomly sample u from $\{1, \dots, k\}$
 - For each router j : $c'_j := \begin{cases} \text{RLWE}_{\widehat{\text{pk}}}(m) & \text{if } j = u, \\ \text{RLWE}_{\widehat{\text{pk}}}(0) & \text{otherwise.} \end{cases}$
 - Randomly sample a from $\{0, \dots, p \cdot n - 1\}$
 - Let A be the binarized a such that $A_x = \text{RGSW}_{\widehat{\text{pk}}}(a_x)$
 - Output $\{c_j\}_{j \in [k]}$ such that $c_j := (c'_j, A)$

ARN with Sender Anonymity construction - II

- We only have to use the placing algorithm for routing.

Rte($\{c_i\}_{i \in [n]}$):

- $c_i := (c'_i, A_i)$
- Output C , the addition of all **HomPlacing**(c'_i, A_i).

- The zero-encrypted messages do not affect the algorithm.

ARN with Sender Anonymity construction - III

- Collectively run the multi-party decryption method.

Dec(user secret keys, C):

- $\mathbf{m}^* \leftarrow \text{MP.Dec}(\text{user secret keys}, c^*)$ where $c^* \in C$.
- Output multiset of all non-zero $\mathbf{m}^*$

- Non-zero messages are forwarded to every user.

ARN with Receiver Anonymity

ARN with Receiver Anonymity

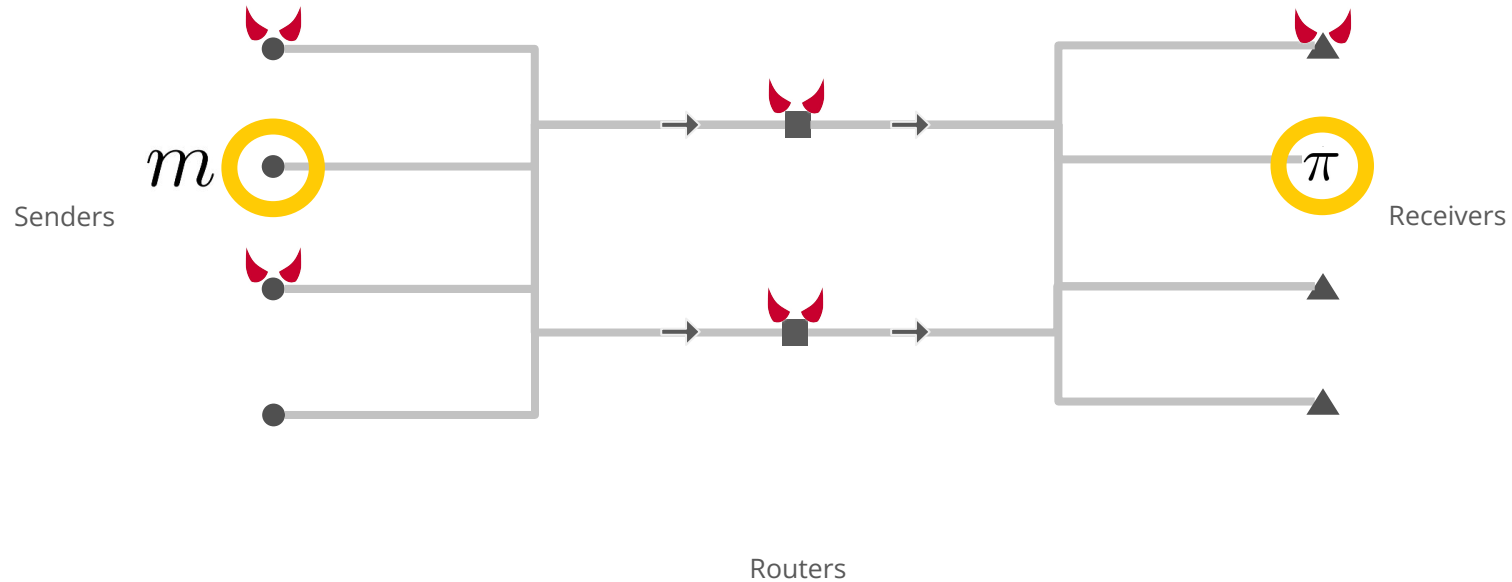
- Sender Anonymity seems very lacking.
- However, it is very useful in creating ARN with Receiver Anonymity.
- Reminder:
 - Senders don't know the destination.
 - Receivers don't know the sender.
 - A message is only known by its sender and receiver.
- A standard public key seems to fit this perfectly.

Defining public key encryption

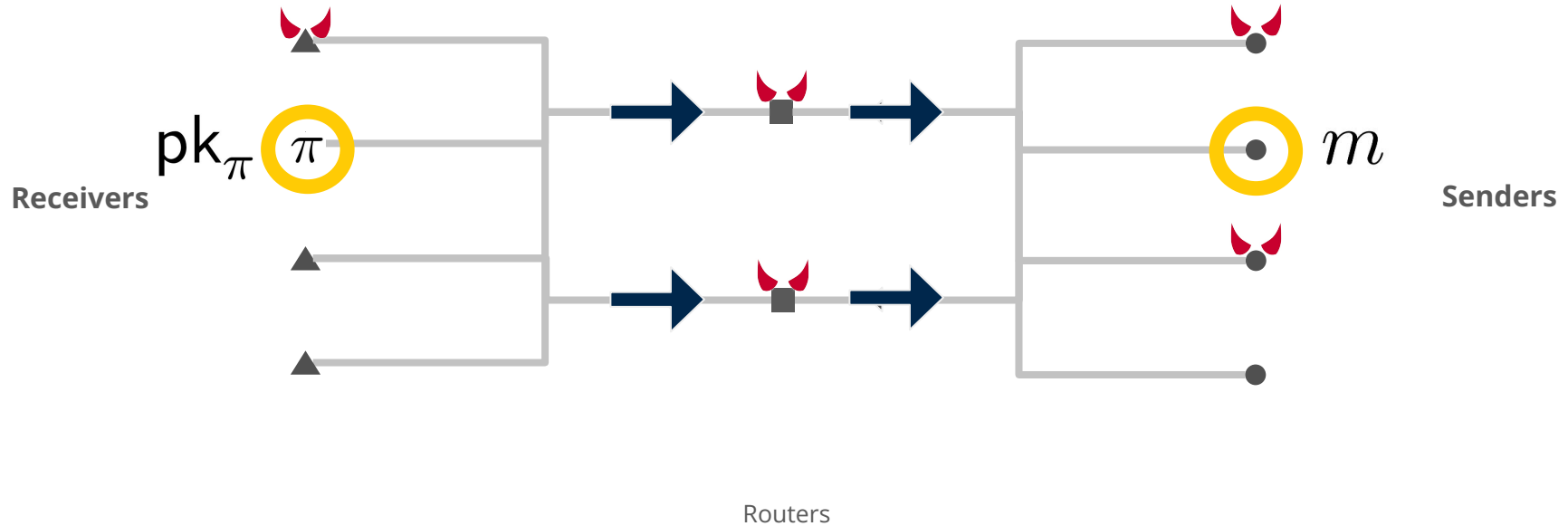
$$\text{Let } \text{PKE} := (\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec}) \text{ where } \left\{ \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \mathbf{Gen}(1^\lambda), \\ c \leftarrow \mathbf{Enc}(\text{pk}, m), \\ m \leftarrow \mathbf{Dec}(\text{sk}, c). \end{array} \right\}$$

A user's public key should not have identifiable information!

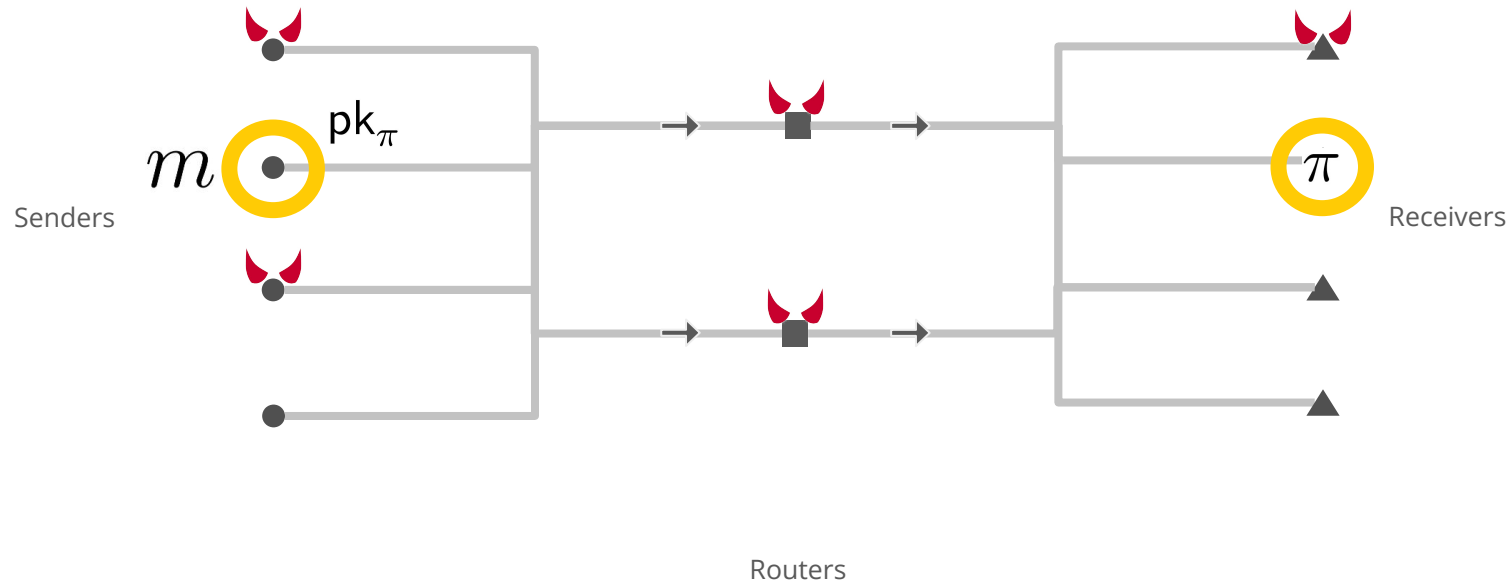
Receiver Anonymity example



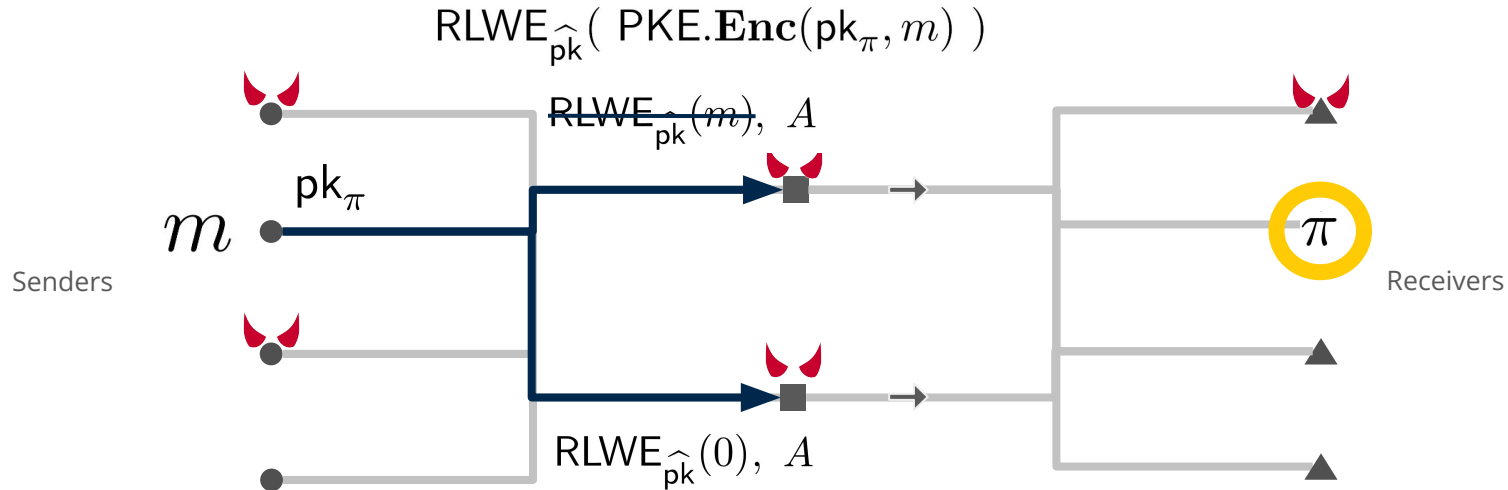
Receiver Anonymity example



Receiver Anonymity example



Receiver Anonymity example



$$\forall x \in [\log p \cdot n], A_x = RGSW_{\hat{pk}}(a_x) \text{ where } a \in \{0, 1\}$$

ARN with Receiver Anonymity construction - I

- **Setup**($1^\lambda, n, k$):
 - Output $\widehat{\text{pk}}^*$ and user secret keys from $\text{MP.Setup}(1^\lambda, n, k)$.
 - For each receiver ℓ : $\text{pk}_\ell, \text{sk}_\ell \leftarrow \text{PKE.Gen}(1^\lambda)$
 - Run the ARN with receiver anonymity protocol with sender/receiver role reversal to distribute PKE public keys.
 - Output $\widehat{\text{pk}}$ and user secret keys where $\widehat{\text{pk}} := (\widehat{\text{pk}}^*, \text{pk})$

ARN with Receiver Anonymity construction - II

- $\mathbf{Enc}(\widehat{\mathbf{pk}}, m, \pi)$:
 - $\widehat{\mathbf{pk}} := (\widehat{\mathbf{pk}}^*, \mathbf{pk})$ and $c^* := \mathbf{PKE}.\mathbf{Enc}(\mathbf{pk}_\pi, m)$.
 - Output ARN with sender anonymity's $\mathbf{Enc}(\widehat{\mathbf{pk}}^*, c^*, \pi)$
- $\mathbf{Rte}(\{c_i\}_{i \in [n]})$: Output ARN with sender anonymity's routing algorithm.
- $\mathbf{Dec}(\text{user multi-party secret keys}, C_j)$: Output sender anonymity's decryption.

Roadmap

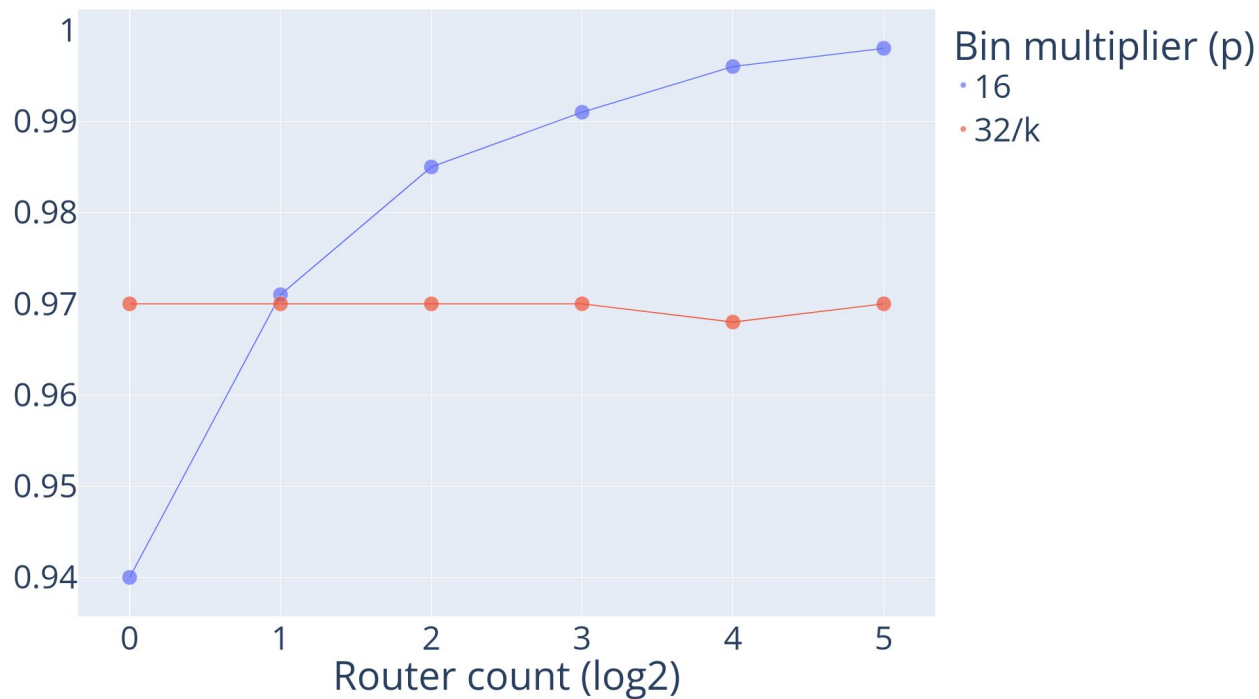
- Preliminaries and Overview
- Security Definitions
- Relevant Schemes/Algorithms
- Our Construction
- **Results and Benchmarks**
- Conclusion

The bin multiplier p

- Failure is largely dependent on it.
- We show the probability of success for two values:

$$p \in \{ 16, 32/k \}$$

Success probabilities per the bin multiplier



Implementation details

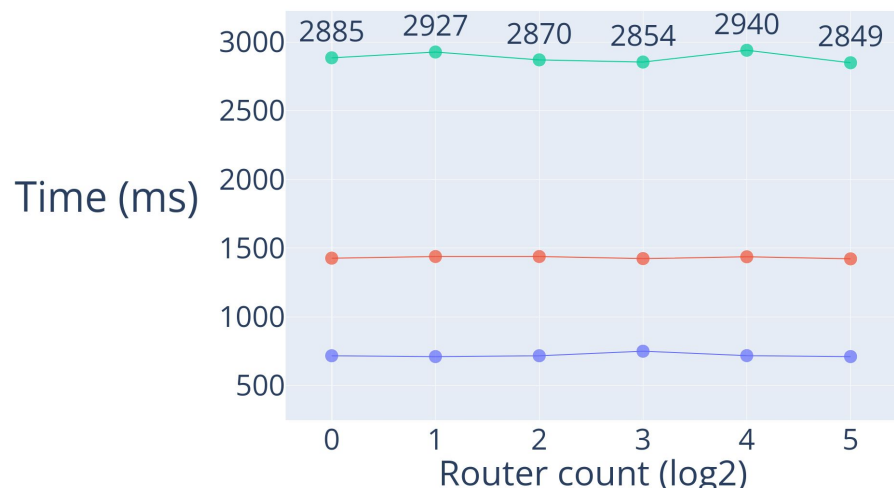
- Using the Lattice-Estimator library, yields **115 bits of security**.
- Built in Rust with the TFHE-rs library.
- Intel i5-14400
- 32GB of RAM

Benchmark details

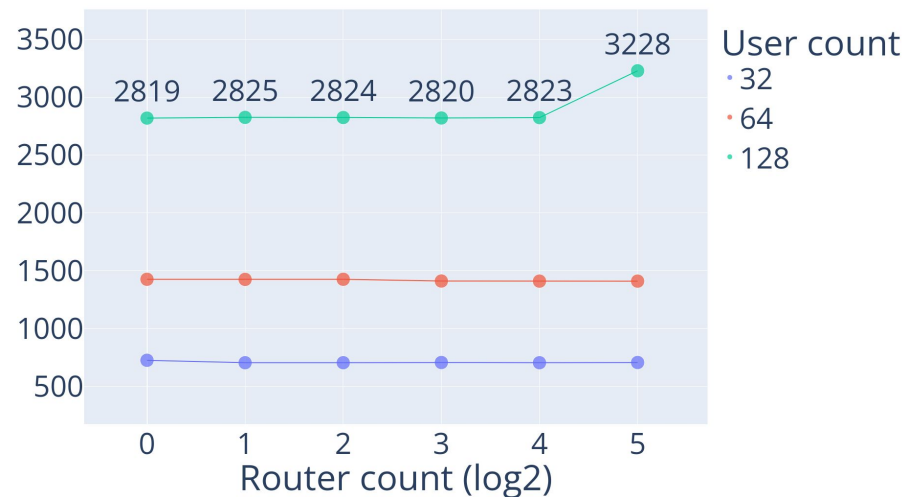
- We show computation times for
 - $p \in \{ 16, 32/k \}$
 - and $n \in \{32, 64, 128\}$.
- We give a non-direct comparison to sPAR

Setup times for $p \in \{ 16, 32/k \}$

$p = 16$

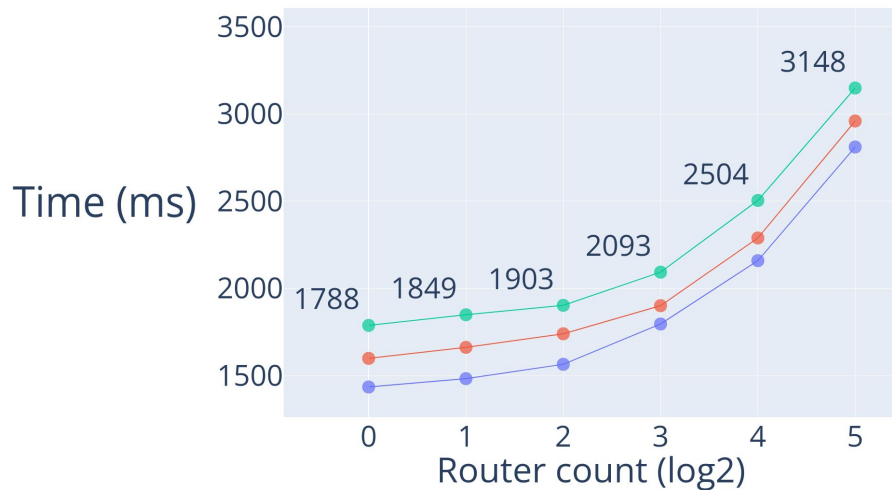


$p = 32/k$

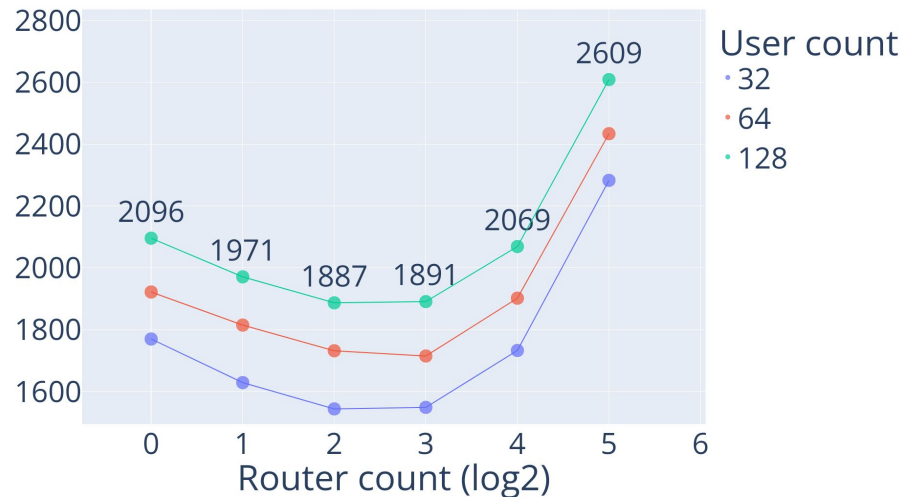


Encryption times for $p \in \{ 16, 32/k \}$

$p = 16$

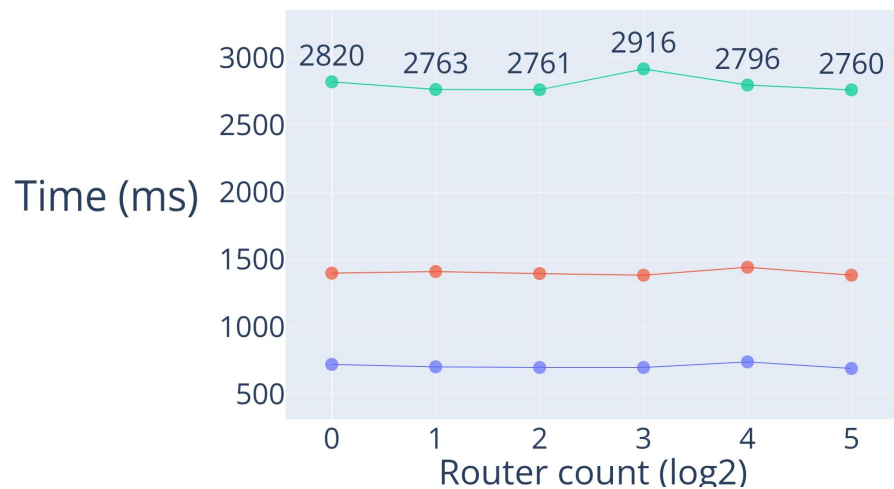


$p = 32/k$

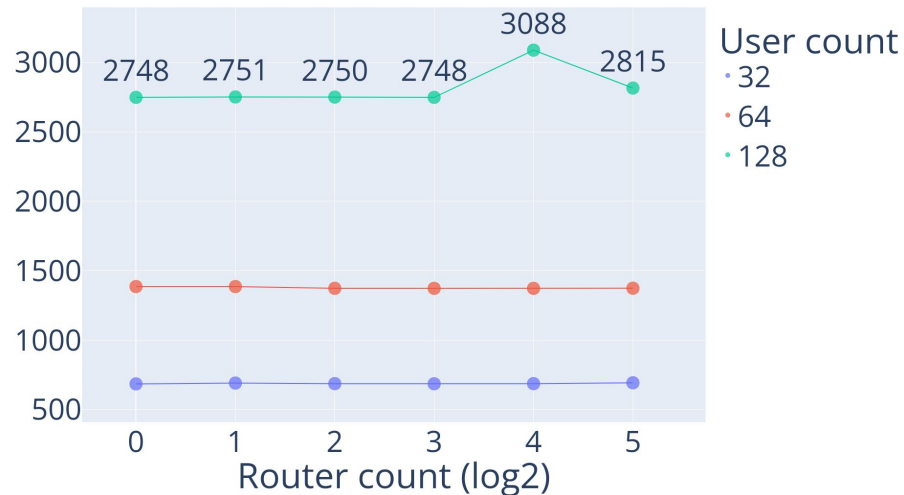


Decryption times for $p \in \{ 16, 32/k \}$

$p = 16$

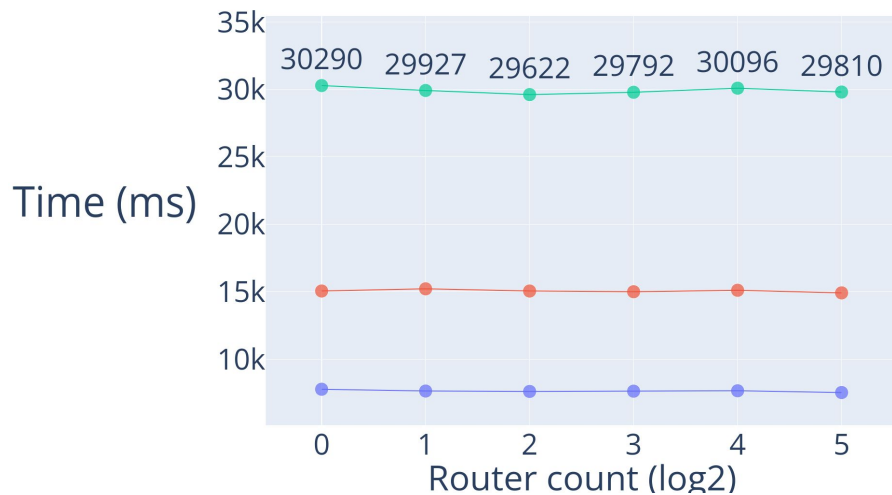


$p = 32/k$

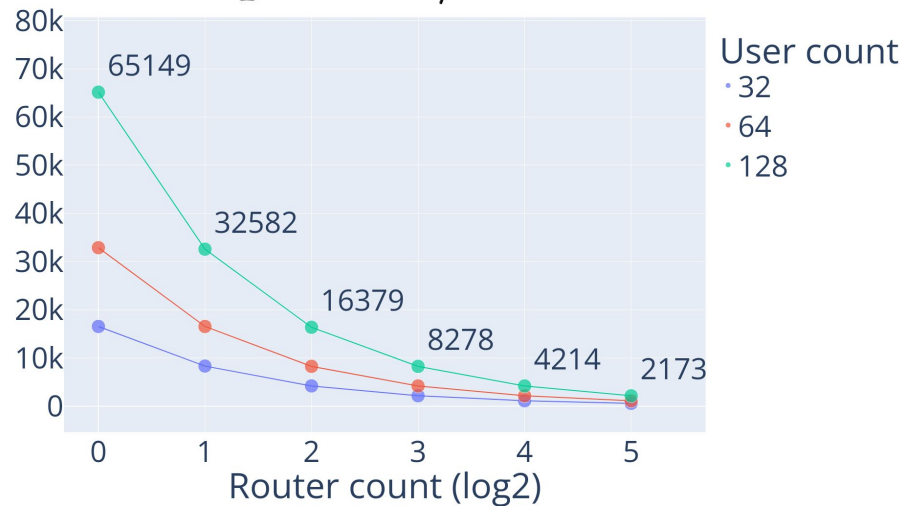


Router times for $p \in \{ 16, 32/k \}$

$p = 16$



$p = 32/k$



Roadmap

- Preliminaries and Overview
- Security Definitions
- Relevant Schemes/Algorithms
- Our Construction
- Results and Benchmarks
- **Conclusion**

Another detour for the bin multiplier p

- This is a significant departure from sPAR.
- Allows a smaller bin multiplier and smaller probability of failure.
- Essentially, it allows for $p = 1/k$
- However, there are some complications.

Final remarks

- The scenario in which some routers fail, a subset of messages still get transmitted.
- This opens the door to users picking a subset of routers
- We achieved an improvement in performance

Thank you

Questions?

References - I

- Craig Gentry, Amit Sahai, and Brent Waters. “Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based”. In: Advances in Cryptology – CRYPTO 2013, pp. 75–92. isbn: 978-3-642-40041-4.
- Vadim Lyubashevsky, Chris Peikert, and Oded Regev. “On ideal lattices and learning with errors over rings”. In: EUROCRYPT’10. French Riviera, France: Springer-Verlag, 2010, pp. 1–23. isbn: 3642131891. doi: 10.1007/978-3-642-13190-5_1. url: https://doi.org/10.1007/978-3-642-13190-5_1.
- Jeongeun Park. “Homomorphic Encryption for Multiple Users With Less Communications”. In: IEEE Access 9 (2021), pp. 135915–135926. doi: 10.1109/ACCESS.2021.3117029.
- Shi, E., Wu, K. (2021). Non-Interactive Anonymous Router. In: Canteaut, A., Standaert, FX. (eds) Advances in Cryptology – EUROCRYPT 2021. EUROCRYPT 2021. Lecture Notes in Computer Science(), vol 12698. Springer, Cham. https://doi.org/10.1007/978-3-030-77883-5_17
- Roger Dingledine, Nick Mathewson, and Paul Syverson. “Tor: The Second-Generation Onion Router”. In: 13th USENIX Security Symposium (USENIX Security 04). San Diego, CA: USENIX Association, Aug. 2004. url: <https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router>.

References - II

- Ilaria Chillotti et al. “TFHE: Fast Fully Homomorphic Encryption Over the Torus”. In: 33.1 (Jan. 2020), pp. 34–91. issn: 0933-2790. doi: 10.1007/s00145-019-09319-x. url: <https://doi.org/10.1007/s00145-019-09319-x>.
- Christian Mouchet et al. “Multiparty Homomorphic Encryption from Ring-Learning-with-Errors”. In: Proceedings on Privacy Enhancing Technologies 2021 (Oct. 2021), pp. 291–311. doi: 10.2478/popets-2021-0071.
- Kelong Cong et al. “SortingHat: Efficient Private Decision Tree Evaluation via Homomorphic Encryption and Transciphering”. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. CCS ’22. Los Angeles, CA, USA: Association for Computing Machinery, 2022, pp. 563–577. isbn: 9781450394505. doi: 10 . 1145 / 3548606 . 3560702. url:<https://doi.org/10.1145/3548606.3560702>.
- Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data. <https://github.com/zama-ai/tfhe-rs>. 2022.
- Martin R. Albrecht, Rachel Player, and Sam Scott. “On the concrete hardness of Learning with Errors”. In: Journal of Mathematical Cryptology 9 (2015), pp. 169–203. url: <https://api.semanticscholar.org/CorpusID:86408>.